

← Go Back

Hardware

Portenta Vision Shield

Tutorials

Blob Detection with Portenta and OpenMV

Saving Bitmap Camera Images to the SD Card

Connect Portenta Vision Shield to TTN with LoRa® Technology

Creating a Basic Face Filter With OpenMV

Training a Custom Machine Learning Model for Portenta H7

Using the Portenta Vision Shield - Ethernet with Arduino IDE

Using the Portenta Vision Shield - Ethernet with OpenMV

Getting Started With the Portenta Vision Shield Camera

Voice Commands With The Arduino Speech Recognition Engine

Connecting to The Things Network Using OpenMV

Portenta Vision Shield User Manual

Home / Hardware / Portenta Vision Shield / Creating a Basic Face Filter With OpenMV

Creating a Basic Face Filter With OpenMV

In this tutorial you will build a MicroPython application with OpenMV, to use the Portenta Vision Shield to detect faces and overlay them with a custom bitmap image.

Author: Sebastian Romero, Lenard George · Last revision: 09/25/2024

ON THIS PAGE

Overview

- Goals +
- The Haar Cascade Algorithm
- Instructions +
- Conclusion +
- Troubleshooting +

Overview

In this tutorial you will build a MicroPython application with OpenMV, to use the Portenta Vision Shield to detect faces and overlay them with a custom bitmap image. Think of it as building your own camera filter that puts a smile on every face it detects. This tutorial is based on the face detection example that comes with the OpenMV IDE.

Goals

- How to use the OpenMV IDE to run MicroPython on Portenta
- How to use the built-in face detection algorithm of OpenMV
- Copying files to the internal Flash of the Portenta

Help

Required Hardware and Software

Portenta H7

Portenta Vision Shield

USB-C® cable

Arduino IDE 1.8.10+ or

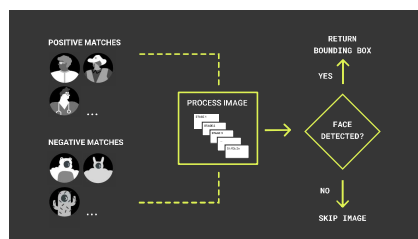
Arduino IDE 2.0+

Portenta Bootloader Version
20+

OpenMV IDE 2.6.4+

The Haar Cascade Algorithm

By harnessing the power of machine vision algorithms, objects can be detected in a camera stream. Those algorithms can be trained to detect the desired type of object. In this tutorial, you will use a machine learning based approach called Haar Cascade to detect faces.



This approach uses a cascade algorithm that has multiple stages, where the output from one stage acts as additional information for the next stage in the cascade. The different stages are responsible for detecting edges, lines, contrast checks and calculating pixel values in a given image. Larger areas of

numerous and smaller area checks in later stages. The Haar Cascade function provided by OpenMV allows to specify the amount of stages. Fewer stages make the detection faster, while leading to more false positives.

The built-in Haar Cascade model for faces was trained with hundreds of images containing faces that are labeled as such and images that do not contain faces labeled differently. That allows the algorithm to distinguish such images after it is being trained.

Instructions

Creating the Face Detection Script

For this tutorial, you will be using the OpenMV IDE along with the OpenMV firmware on your Portenta H7 to build the face detection script. If this is your first time using the Portenta Vision Shield and OpenMV, we recommend you to take a look at the "Configuring the Development Environment" section inside the [Blob Detection tutorial](#) to configure the development environment.

1. The Basic Setup

Attach your Portenta Vision Shield to your Portenta H7 and open the **OpenMV** Editor. For this tutorial, you will create a new script that is based on the face detection

"New File" button in the toolbar on the left side and save it as **face_detection.py**.

2. Importing the Modules

The script starts by importing the `sensor`, `image` and `time` modules for handling the camera sensor, using machine vision algorithms and time tracking functions.

```
1 import sensor # Import t
2 import image # Import mo
3 import time # Import mod
```

3. Preparing the Sensor

The next step is to calibrate the camera sensor to achieve the best results using the `sensor` module. You can use the `set_contrast()` function to set the contrast of the sensor to its highest value (3). This can help the algorithm identifying lines and edges more easily. `set_gainceiling()` controls the amplification of the signal from the camera sensor including any associated background noise. For maximizing the detection success rate, it is recommended to set the camera frame size to `HQVGA`.

```
1 # Sensor settings
2 sensor.set_contrast(3)
3 sensor.set_gainceiling(10)
4 sensor.set_framesize(sensor.FRAMESIZE_160x120)
5 sensor.set_pixformat(sensor.RGB565)
```

4. Finding the Face Features

OpenMV provides a Haar Cascade class ready to be used with the Portenta Vision Shield's camera.

The function

```
image.HaarCascade(path, number  
of stages)
```

is used to load a Haar Cascade model into memory. The `path` parameter can be used to either specify the location of a custom Haar Cascade model or to use the built-in `frontalface` model. The `stages` parameter is used to specify the desired Haar Cascade stages.

 Remember: Fewer stages make the detection faster while leading to more false positives.

```
1 face_cascade = image.HaarCascade('frontalface', 5)
2 print(face_cascade) # Print the loaded cascade
```

5. Displaying a Bitmap Image

Once you know the location of the

choice. OpenMV currently supports bmp, pgm or ppm image formats. Image formats with an alpha layer such as PNG are not supported yet.

In this tutorial you will use a preloaded smiley image in the monochrome **Portable Bitmap Image** (.pbm) format. This format consists of a matrix of zeroes and ones denoting black and white pixels. 1 stands for a black pixel, 0 for a white one. If you want to create your custom image, make sure you save it in one of the supported bitmap formats (bmp, pgm or ppm). You may use an image editor of your choice which supports exporting images in one of these formats. For this tutorial Adobe Photoshop was used.

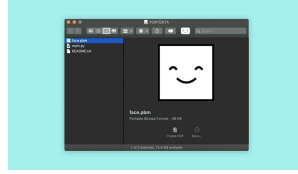
Connect your Portenta board to your computer if you have not done so yet. Make sure you are running the OpenMV firmware on the Portenta. If you have not installed the OpenMV firmware yet, take a look at the "Configuring the Development Environment" section which explains how to proceed in that case.

Download [this file](#) containing the smiley bitmap and copy it to the Flash drive that was mounted when you connected the Portenta running the OpenMV firmware.

The on-board memory gets mounted as a Flash drive.



Therefore this will work even if you have not inserted an SD card.



Copy the bitmap image
to the Flash drive

Load the image into a variable called `faceImage` using the `Image()` function from the `image` module. The initial slash refers to the root directory of the Flash drive. In order to use the image as an overlay to the camera stream, instead of directly displaying it, set the `copy_to_fb` to `False` such that it does not get copied into the frame buffer automatically.

```
1 faceImage = image.Image(
```

Before drawing the image on top of the camera stream you need to figure out the scale ratio to match the detected face size in the camera stream. The provided bitmap image comes in a 128x128 px resolution. You can calculate the correct scale ratio with the following formula:

```
1 faceX = boundingBox[0]
2 faceY = boundingBox[1]
3 faceWidth = boundingBox[2]
4
5 # Calculates the scale ratio
6 scale_ratio = faceWidth
```

You can then draw the scaled bitmap image on top of the camera image using the `draw_image` function:

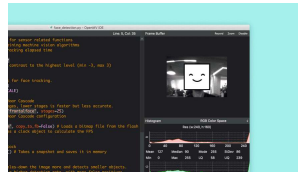
```
1 # Draws the bitmap on top of the camera image
2 cameraImage.draw_image(f
```

6. Uploading the Script

Let's program the Portenta with the complete script and test if the algorithm works. Copy the following script and paste it into the new script file that you created.

```
1 import sensor # Import
2 import image # Import
3 import time # Import m
4
5 sensor.reset() # Reset
6 sensor.set_contrast(3)
7 sensor.set_gainceiling
8
9 # HQVGA and GRAYSCALE
10 sensor.set_framesize(s
11 sensor.set_pixformat(s
12
13 # Load the built-in fr
14 # By default this will
15 face_cascade = image.H
16 print(face_cascade) #
17
18 faceImage = image.Imag
19 clock = time.clock() #
20
21 while (True):
22     clock.tick() # Adv
23     cameraImage = sens
24
25     # Find objects.
26     # Note: Lower scal
27     # Higher threshola
28     boundingBoxes = ca
29
```


Click on the "Play" button at the bottom of the left toolbar. Point the camera on the Portenta Vision Shield towards your face and check if the Portenta can detect it. Once it detects your face, it should be covered with a smiley.



Copy the bitmap image
to Flash drive

Conclusion

In this tutorial you learned how to use OpenMV's built-in face detection algorithm which is based on Haar Cascade. Furthermore, you learned how to copy a file to the internal flash and how to load an image from the Flash into the memory. You have also learned how to draw an image on top of a snapshot from the camera stream.

Next Steps

The [HaarCascade](#) class provided by OpenMV can also detect other facial features such as eyes. For example you could tweak your **face_detection.py** script to detect your eyes simply by changing the `path` parameter from `frontalface` to `eye` which is also a built-in model. Go ahead and replace the following line in your script and try to figure out how to overlay your eyes with a bitmap

```
1 face_cascade = image.Haa
```

Troubleshooting

Face Detection Issues

If OpenMV cannot detect your face, try moving the camera further away or position yourself in front of a wall or another plain background.

Bitmap Loading Issues

If you have troubles loading a custom bitmap image, try with the pbm format and try scaling it down to a smaller size such as 128x128 pixels.

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub repository . If you see anything wrong, you can edit this page here .	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?

